

Skill-Manager User Manual

Operating the m3c Skill Lifecycle Management stack

m3c · Scalytics — KuP Berlin training

2026-05-07

Contents

Skill-Manager – User Manual	1
0. Read this first	1
1. Quick start by role	2
2. The system, in one diagram	4
3. Concepts you cannot skip	5
4. The five lifecycle flows	7
5. Role playbooks (detailed)	12
6. Decision tree: which command should I run?	19
7. Maturity model	21
8. Exit codes	21
9. Cookbook	23
10. Failure modes & remediation	26
11. References	26

Skill-Manager – User Manual

A trustworthy supply chain for living skills, with field-feedback mechanisms. This manual tells you, **by role**, what to do – what to type, what to expect, what to avoid. The companion [SKILLCTL-MANUAL.md](#) documents every skillctl command.

0. Read this first

A **skill** is a directory of instructions, scripts, prompt templates, and references that an agent (Claude Code) loads and executes. Skills live at:

Tier	Where	Who installs
project	<repo>/. <code>claude</code> /skills/<name>/	repo owner
user	~/. <code>claude</code> /skills/<name>/	end-user
plugin	~/. <code>claude</code> /plugins/cache/**/skills/<name>/	plugin marketplace

Project beats user beats plugin (Claude Code’s own resolution rule). Without governance, every skill is **arbitrary code at the LLM’s privilege level**. The Skill Manager is the m3c capability that turns those directories into a signed, attested, audited supply chain.

Six things you should know before you touch the system:

1. The unit of trust is a **skill bundle** (<name>-<version>.skb) – a deterministic gzipped tar whose SHA-256 is the canonical bundle_digest.
2. The chain layers **three signatures**: author, registry, governance – each detached, named after the digest, named <digest>.author.sig / .registry.sig / .governance.sig.
3. The **registry** is API-only (aims-core/flask/modules/skill_registry/). The **CLI** is skillctl. The **UIs** are skillprofile (your personal skills) and the **CISO Console** (/v2/ciso?tenant=<id>).
4. Every state-changing operation has a **numbered exit code** (SPEC-0188 §11 + §17-§39 across SPEC-0201/-0202). Scripts and pre-commit hooks should branch on exit codes, not parsed log lines.
5. **Trust attaches at our boundary, never upstream.** Public hubs are discovery surfaces; bytes only enter the registry through skillctl import-public -> propose -> reviewer attestation (SPEC-0201).
6. **Every invocation is gated.** Even after install, a skill runs under a short-lived **capability token** (SPEC-0202) that bounds egress, subprocess, file I/O, and runtime. Refusals are audited.

If you only read one thing else: read SECTION 4 – The Five Lifecycle Flows. Everything else is reference.

1. Quick start by role

Find your role. Run the three commands. Read the linked playbook.

1.1 Author / Skill-Steward

You write skills. You ship them.

```
# 1) Verify your environment
skillctl --help                    # CLI present
ls ~/.claude/skillctl-keys/author.key  # keypair present (else: `skillctl keygen`

# 2) Iterate on a skill, then propose
skillctl propose <skill-name> --intent yellow --rationale "<what changed>"

# 3) Watch the queue
open https://aims.example.com/v2/skills/admin  # your bundle awaits a reviewer
```

Detailed playbook: §5.1 Author.

1.2 Reviewer

You sign the binding governance verdict.

```
# 1) See what's awaiting review
open https://aims.example.com/v2/skills/admin  # tab: Bundles -> filter "no_governance"

# 2) Read the bundle, run the Activation Gate (SPEC-0130 §5.1)
skillctl install <name>@<version> --home /tmp/review-sandbox --allow-yellow
diff -ur /tmp/review-sandbox/.claude/skills/<name>/ <expected>

# 3) Sign the verdict
skillctl attest <bundle-digest> --level green --rationale "<why>" \
  --reviewer-id id:reviewer@<org> --key ~/.claude/skillctl-keys/reviewer.key
```

Detailed playbook: §5.2 Reviewer.

1.3 CISO (Tenant Security Officer)

You authorize what runs inside *your* tenant.

```
# Everything happens in the console
open https://aims.example.com/v2/ciso?tenant=<your-tenant>
# Tabs you'll use:
#   Bundles           -- the inventory
#   Attestation queue -- Approve · Block · Escalate
#   Data sources      -- per-(skill, source) [authorize|deny|pending] chips
#   Runtime (SPEC-0202) -- live tokens, [x] revoke, [a] attenuate
#   Audit             -- quarterly export PDF/CSV
```

Detailed playbook: §5.3 CISO.

1.4 Registry Operator

You own keys, identities, per-tenant infrastructure.

```
# Provision a tenant
cd m3c-tools-maintenance/INFRA/skill-registry/
./bootstrap.sh --tenant <tenant-id> # GCS bucket + Firestore namespace + Secret

# Rotate the registry key (overlap-publish)
open CISO-WORK/RUNBOOK/skill-registry-key-rotation.md # follow this verbatim

# Register an author identity (v1 manual)
curl -X POST $REG/api/skills/identities \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -d '{"id":"id:author@org","ed25519_pubkey":"<base64>","tier":"user"}'
```

Detailed playbook: §5.4 Registry Operator.

1.5 Tenant Admin

You configure the per-tenant runtime: policy, SLAs, role bindings.

```
# Edit the tenant config
$EDITOR tenants/<tenant-id>/config.yaml # then commit
# Bind a CISO
curl -X POST $REG/api/tenants/<tenant-id>/role-bindings \
  -d '{"role":"tenant_ciso","identity":"id:ciso@<org>}'
```

Detailed playbook: §5.5 Tenant Admin.

1.6 End-User / Skill Consumer

You install verified skills and run them.

```
# 1) Pin trust roots (one-time)
skillctl trust add --registry https://aims.example.com/api/skills \
  --pubkey ~/.config/m3c/skill-trust-roots/<reg>.pub --id <reg-id>

# 2) Install
skillctl install didactic-session@1.0.0 --tenant <your-tenant>
```

```
# 3) Run (envelope-bounded -- SPEC-0202)
skillctl run didactic-session
```

Detailed playbook: §5.6 End-User.

1.7 Auditor

You read, you reconstruct, you flag.

```
# 1) Pull a quarterly audit export from the CISO console
open https://aims.example.com/v2/ciso?tenant=<id>&tab=audit
```

```
# 2) Reconstruct a single skill's chain
```

```
curl $REG/api/skills/by-digest/<digest>
```

```
curl $REG/api/skills/<digest>/signatures
```

```
curl $REG/api/skills/<digest>/attestations
```

```
curl $REG/api/skills/audit-log?bundle_digest=<digest>
```

```
# bundle row
```

```
# author + registry sigs
```

```
# all governance attestations
```

```
# chronology
```

```
# 3) Compare to local install
```

```
skillctl audit
```

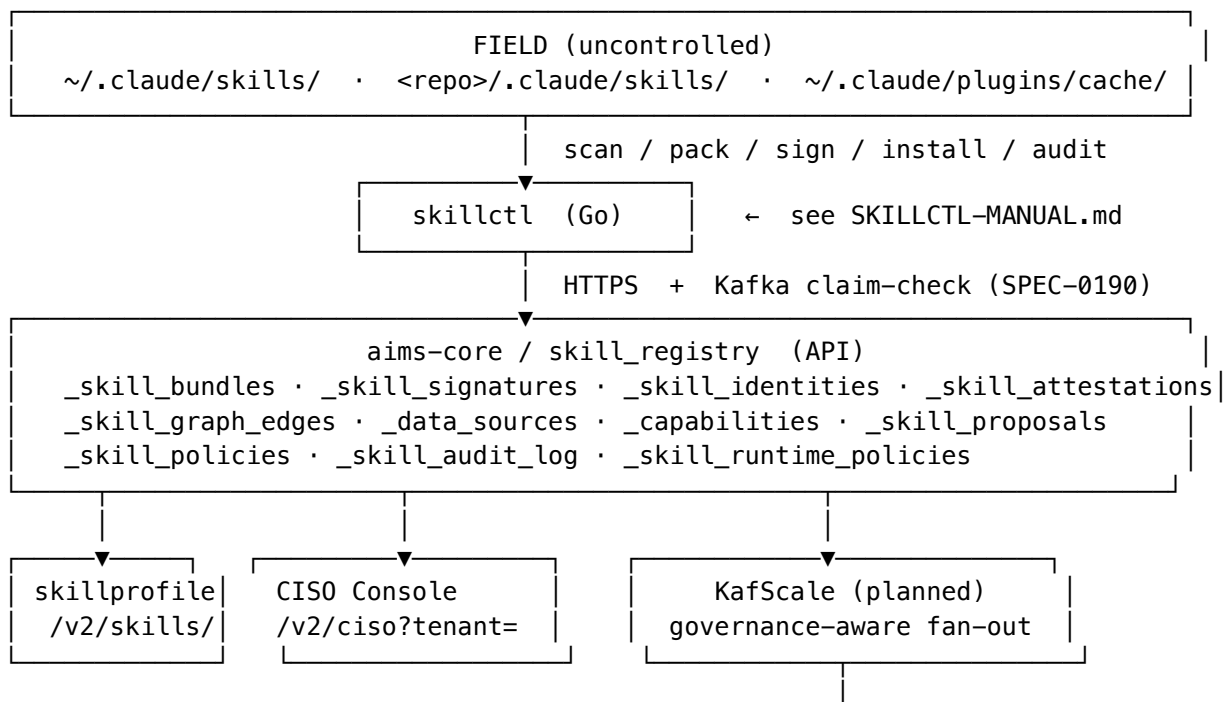
```
# OK / UNVERIFIED / BROKEN
```

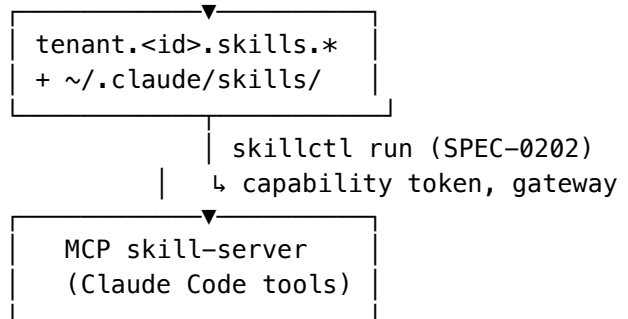
Detailed playbook: §5.7 Auditor.

1.8 m3c Consultant (delivery role)

You take a customer Bronze->Silver->Gold (see §7 Maturity model). Your authority is operational only – you act as Author for training corpora and never as Reviewer or CISO of the customer's production skills.

2. The system, in one diagram





The **load-bearing files on every operator's laptop**:

Path	What	Owner
~/.claude/skill-trust-roots.yaml	Pinned registry public keys (which keys you accept)	End-User pins; Operator distributes
~/.claude/skillctl-keys/author.key	Author signing key (PEM PKCS#8 mode 0600)	Author
~/.claude/skillctl-keys/reviewer.key	Reviewer signing key (Q3-2026 stays registry-on-behalf-of in v1)	Reviewer
~/.claude/skill-import-policy.yaml	Source policy for import-public (SPEC-0201)	CISO/Operator
~/.cache/m3c/imports/<sha256>	Stage area for upstream imports (post-airlock, pre-propose)	Author
~/.config/m3c/skill-keys/	Suggested keypair location from skillctl keygen	Anyone holding a key

If any of those files leak to git, treat as an incident. The pre-commit git leaks scan from /security-scan catches the .priv extension by default.

3. Concepts you cannot skip

3.1 Bundle anatomy

A .skb is a **gzip-compressed tar with deterministic normalization** (SPEC-0188 §3.1). Two skillctl pack runs over the same input produce byte-identical bytes.

```

didactic-session-1.0.0.skb
|-- bundle.json           ← manifest
|-- CHECKSUMS             ← sha256 per file
|-- SKILL.md              ← the skill itself
|-- helpers/
|-- profiles/
|-- scripts/              ← mode 0755; everything else 0644

```

Determinism rules: lexicographic order, mtime = 1970-01-01T00:00:00Z, tar PAX format with no extended headers, gzip header zeroed (gzip --no-name semantics), no .DS_Store / .git at root. built_at in bundle.json is also Unix epoch – wall-clock would break determinism; the real timestamp lives in the registry attestation.

3.2 The bundle.json manifest

```
{
  "schema": "m3c-skill-bundle/v1",
  "name": "didactic-session",
  "version": "1.0.0",
  "summary": "Scaffold a live training session for a specific role-track.",

  "source_repo": "kamir/.claude",
  "source_commit": "",
  "source_path": ".claude/skills/didactic-session",

  "author_governance_intent": "yellow",
  "author_governance_rationale": "Generates files; no destructive ops.",

  "depends_on": [],
  "supersedes": null,
  "derived_from": null,

  "intent": {
    "summary": "...",
    "side_effects": ["fs:write", "llm:call"],
    "destructive": false,
    "network": false,
    "subprocess": ["pandoc", "git"],
    "claims": ["skill:document-generation"]
  },
  "data_dependencies": [
    {
      "id": "ds:filesystem/cwd", "kind": "local_fs",
      "access": "write", "scope": "<cwd>/decks/**",
      "reason": "Write scaffolded decks."
    }
  ],

  "runtime_envelope_required": true,

  "imported_from": null,

  "bundle_digest": "sha256:cd1d5c79...",
  "built_at": "1970-01-01T00:00:00Z",
  "built_by": "skillctl/1.0.0"
}
```

Two non-obvious rules:

- `author_governance_intent` is **advisory only**. Verifiers MUST NOT use it for trust decisions; the binding verdict is the most-recent signed governance attestation (SPEC-0188 §3.2 + §4.3). Author intent and signed verdict can disagree; when they do, the attestation wins.
- `intent` and `data_dependencies` are cross-validated by `skillctl pack` (SPEC-0196 §3.3). A destructive skill cannot self-claim (G); a `network=true` skill MUST declare an HTTP `data_dependency`; a write-access dependency MUST come with `intent.destructive=true`. Pack refuses with exit 18 (`intent_inconsistent`).

3.3 The trust chain

Three detached ed25519 signatures alongside the bundle:

```

didactic-session-1.0.0.skb
<digest>.author.sig      ← author signed bundle_digest
<digest>.registry.sig    ← registry signed (digest, author_sig, admission_ts, decision)
<digest>.governance.sig  ← reviewer signed (digest, level, ts, reviewer_id)

```

The verifier (`skillctl install`) checks each signature against pinned roots in `~/.claude/skill-trust-roots.yaml`. Numbered exit codes distinguish failure modes (digest mismatch / author sig invalid / registry not in roots / governance below minimum / depends_on unsatisfied / data dep denied / token expired / etc.). See §8 Exit codes.

3.4 The semantic graph

The trust graph contains **structural** edges (who signed, who admitted, who attested) AND **semantic** edges (what the skill claims to do, what data sources it touches). Stored denormalised in Firestore (`_skill_graph_edges`). Node kinds: Skill, SkillVersion, Identity, Attestation, Capability, DataSource, Role, UserSkillProfile, Package. Edge kinds: signed_by, admitted_by, attested, supersedes, depends_on, claims_capability, reads_from, writes_to, mapped_to, used_by, imported_from. Composite indexes cover all CISO-console queries in O(1) hops; the same shape ports 1:1 to KafGraph when load demands.

3.5 The runtime envelope (SPEC-0202)

Every invocation gets a short-lived signed **capability token** bound to (bundle_digest, tenant_scope, caller_identity, envelope, expires_at=300s). The envelope is the **intersection** of (bundle declaration ⊓ tenant policy ⊓ caller request) – the issuer never grows rights. The gateway shim refuses anything outside the envelope and logs every refusal. The CISO can [×] revoke or [a] attenuate live tokens.

You don't see the envelope until you violate it, and then you see exit 30-39.

4. The five lifecycle flows

Flow diagrams from `ROLES.md` §2, fleshed out into the actual commands at every step.

4.1 Happy path – new skill landing in production

Author –pack+sign→ Registry –admit+attest→ Reviewer –gov-sig→ CISO –tenant-verdict→ End-User

#	Role	Command / action	Produces
1	Author	skillctl propose <name> --intent yellow --rationale "<why>" (runs the SPEC-0194 §6 ten-check ready-to-promote gate, then pack -> sign -> POST /api/skills/bundles)	author.sig + registry row

#	Role	Command / action	Produces
2	Registry Operator (automatic)	Server verifies the author signature against <code>_skill_identities</code> , emits the registry attestation (SPEC-0188 §4.2), writes <code>_skill_bundles</code> , <code>_skill_signatures</code> , <code>_skill_graph_edges</code>	registry.sig + admission edges
3	Reviewer	Open <code>/v2/skills/admin -> Bundles -> click bundle -> read SKILL.md</code> , scripts, intent, <code>data_dependencies -> run Activation Gate (SPEC-0130 §5.1) -> skillctl attest <digest> --level green --rationale "..."</code> <code>--reviewer-id <id> --key <path></code>	governance.sig + attested edge
4	CISO	<code>/v2/ciso?tenant=<id> -> Attestation queue -> Approve/Block/Escalate -> for each data_dependency chip click [Authorize] or [Deny]</code>	tenant verdict + data-source authorization
5	End-User	<code>skillctl install <name>@<version></code> . Verifier checks digest <code>-> author sig -> registry sig -> governance ≥ minimum -> tenant policy -> data deps -> deps installed -> atomic move to ~/.claude/skills/</code> . Then <code>skillctl run <name> for envelope-bounded execution</code>	install record + Kafka invocation events

Tiebreaker: at every step the next role can refuse. Refusal exits with a numbered code; the chain halts.

4.2 Awareness path – scanned local skill becomes registry-aware

This is the SKILLOR-WORK/s1 flow, productized as SPEC-0195. Use it when you have local skills that were never packed (e.g. plugin marketplace installs, hand-rolled scripts).

End-User →scan→ Registry →admit-from-scan→ Auditor →reads chain→ CISO →authorize data deps→ End-User

#	Role	Command / action
1	End-User	<code>skillctl scan --source claude --with-trust --format json > inventory.json</code>
2	End-User	<code>skillctl awareness sync --inventory inventory.json --confirm --default-attest yellow --session "skill-awareness/<host>/<date>"</code>
3	Registry (automatic)	One bundle per scanned skill admitted with <code>gcs_uri = local-aware://<digest></code> . Bundles are <i>known</i> but bytes are not transferable until re-published via the Author path. Per SPEC-0196 §7
4	Auditor	missing-intent skills get sentinel in- <code>tent.side_effects=["UNKNOWN"]</code> + <code>unverified_intent</code> marker edge
5	CISO	Reads the new admissions in the audit log; flags any skill with UNKNOWN intent for follow-up
6	End-User	Sees new entries in the verdict queue with needs declaration badges. Until authorized, downstream consumers see pending chips
		<code>skillctl awareness verify --session "<tag>"</code> reads back per-session admissions for confirmation

A full worked example (with reset-and-replay, snapshots, and verification) lives in [SKILLOR-WORK/s1/USER-MANUAL.md](#).

4.3 Demotion path – Reviewer revokes attestation

Reviewer →new-attest (R)→ Registry →republish current_gov→ Tenant Admin →notify→ End-User loader

#	Role	Command / action
1	Reviewer	<code>skillctl attest <digest> --level red --rationale "<why>" --reviewer-id <id> --key <path></code> (multiple attestations allowed; most-recent qualified one wins)
2	Registry (automatic)	Updates <code>_skill_attestations</code> ; the graph's attested edge resolves to the new verdict on next read
3	Tenant Admin (notification)	Receives an SLA-tracked notification (SPEC-0192 §7) for any tenant bundle whose <code>current_governance</code> dropped below the tenant's <code>governance_minimum</code>
4	CISO	Either accept (block propagates) or override (re-attest (G) tenant-scoped)
5	End-User loader	Next <code>skillctl install</code> sees governance below minimum (exit 14) and refuses. Already-installed copies on disk still work – <code>skillctl audit</code> flags them on the next sweep; <code>--cleanup</code> removes them

4.4 Rotation path – Author rotates key

Author `--new-key` → Registry `--register identity v2` → Reviewer `--re-sign new bundle` → End-User upgrade

#	Role	Command / action
1	Author	<code>skillctl keygen --out ~/.claud/skillctl-keys/author-v2</code> . Keep old key marked retired.
2	Registry Operator	POST <code>/api/skills/identities</code> with the new pubkey; identity row carries both pubkeys during the overlap window with retired:<date> on the old one
3	Author	Re-sign current bundles with new key; push via <code>skillctl propose</code>
4	Reviewer	Re-attest only if policy requires fresh attestation per key (most don't – attestation binds to <code>bundle_digest</code> , not signing key)

#	Role	Command / action
5	End-User	Pinned ~/.claude/skill-trust-roots.yaml accepts both keys during overlap. After retired date, old-key bundles fail verification on re-install. Existing installs keep working until uninstalled

The same overlap pattern applies to **registry-key rotation** – see CISO-WORK/RUNBOOK-skill-registry-key-rotation.md.

4.5 Author-key revocation (SPEC-0198)

When an author key is compromised:

#	Role	Command / action
1	Author (or Operator if author is unreachable)	POST /api/skills/identities/<id>/revoke with rationale and effective timestamp
2	Registry	Updates identity row; emits revoked graph edges for every bundle signed by the compromised key; publishes m3c.<env>.skill_identities.revoked Kafka event
3	KafScale (when active)	Republishes per-tenant; downstream skillctl install short-circuits with exit 13/19
4	End-User running skillctl audit	Affected skills move to BROKEN; --cleanup removes them
5	Auditor	The full chain – fetch sha256, scan report digest, importer signature, reviewer attestation, CISO verdict, revocation row – survives in _skill_audit_log for forensic review

4.6 Upstream import path (SPEC-0201)

When a skill comes from a public hub (skillhub.club, claudeskills.info, lobehub.com/skills, etc.), it MUST cross the airlock before reaching the registry:

```
discover  -> airlock      -> re-pack/sign -> attest    -> install
hub UI/API skillctl      skillctl pack  reviewer   skillctl install
import-public          + sign          attests    (full chain)
```

1) Discover on a hub UI; copy the slug

```
skillctl import-public skillhub://didactic-session
```

prints required SHA-256 pin, ...

```
skillctl import-public skillhub://didactic-session@sha256:abc... # stages under ~/.cache/m3c/i
```

```

# 2) Inspect what landed
skillctl import-list
cat ~/.cache/m3c/imports/<sha256>/scan-report.json

# 3) Promote to a real candidate (existing SPEC-0194 flow)
skillctl propose didactic-session \
  --source ~/.cache/m3c/imports/<sha256>/ \
  --derived-from skillhub://didactic-session@sha256:abc... \
  --intent yellow \
  --rationale "imported from skillhub; pending reviewer"

# 4) Reviewer attests (SPEC-0188 §4.3 -- separate human, not the importer)

# 5) Consumers install (SPEC-0188 §7)
skillctl install didactic-session@1.0.0

```

What the airlock enforces (full list in [IMPORT-AIRLOCK.md](#) §“What the airlock enforces”):

1. **Default-deny source policy** (~/.claude/skill-import-policy.yaml) – empty file, nothing imports.
2. **Mandatory content pinning** – every fetch is <scheme>://<ref>@sha256:<hex>.
3. **Explicit license acceptance** – --accept-license <SPDX> or refuse.
4. **Static scanner refuses on:** secret regex hits, os.system / shell-true, file writes outside skill dir, HTTP to non-allowlisted hosts, missing license header, CVE-high deps.
5. **Author-intent capped at (Y) yellow.** Imported bundles cannot self-declare green.
6. **Permanent provenance edge** – (SkillVersion) -[imported_from]-> (ExternalSource host, slug, sha256). CISO console renders it as a chip.
7. **Source-level CISO block** – Tenant CISO can block by host pattern (skillhub.club); affected installs fail with exit 19.
8. **Forbidden upstream MCP servers** – @skillhub/mcp-server is on mcpServers_blocked; skillctl audit rule R-401 flags any settings.json that re-introduces it.

What NOT to do:

- ☒ git clone <hub-repo> into ~/.claude/skills/ – skillctl audit flags UNVERIFIED; release-gate refuses.
- ☒ Add @skillhub/mcp-server to ~/.claude/settings.json – R-401 flags; pre-commit refuses.
- ☒ Run import-public without a pin – the scheme refuses on purpose.
- ☒ Self-attest (G) on an imported bundle – exit 18; reviewer attestation is mandatory.

5. Role playbooks (detailed)

5.1 Author

Daily flow:

1. Edit your skill under ~/.claude/skills/<name>/ or <repo>/.claude/skills/<name>/.
2. Update SKILL.md frontmatter (governance_level, owner, human_checkpoints, context_scope per SPEC-0130 §3).
3. Update bundle.json intent and data_dependencies per SPEC-0196 §3.
4. Bump version in bundle.json (SPEC-0194 §6 check 10 – semver).
5. skillctl propose <name> --intent yellow --rationale "<what changed>" – runs the 10-check ready-to-promote gate (frontmatter complete, governance level set, smoke-test marker

present, semver bumped, no open BUG-NNNN against this skill, intent + data_deps declared, etc.). Gate fails loud (exit 2). Fix and re-run.

6. Bundle is admitted; check the queue: /v2/skills/admin -> Bundles tab.

You may NOT:

- Bind the governance verdict (Reviewer's job).
- Author skills above the access level of your own user account (SPEC-0130 §6.2).
- Edit governance_level post-attestation without re-attestation.
- Author and review the same skill (separation of duties).

Common author errors and fixes:

Symptom	Cause	Fix
skillctl propose exits 2 with "frontmatter incomplete"	SKILL.md missing governance_level / owner / human_checkpoints	Add them to frontmatter, re-run
exits 18 intent_inconsistent	intent.network=true but no HTTP data_dep, OR intent.destructive=false but data_dep[].access=write	Make intent and data_dependencies consistent
exits 18 upstream_intent_capped	Tried --intent green on imported bundle	Use --intent yellow; have a reviewer attest (G) separately
keygen produced wrong filename	Output is <out>.priv and <out>.pub, mode 0600/0644	Move to ~/.claude/skillctl-keys/<name>.{priv,pub}
Sign produces <bundle>.<digest_hex>.author.sig (not just .author.sig)	Correct – signature filename embeds digest	Don't rename; verifier looks for the digest-named file

Files you own:

~/.claude/skillctl-keys/author.key	(PEM PKCS#8 ed25519, mode 0600)
~/.claude/skillctl-keys/author.pub	(PEM SPKI ed25519, mode 0644)
<repo>/~/.claude/skills/<name>/	(the skill source)
~/.claude/skills/<name>/	(your personal skill)

5.2 Reviewer

Daily flow:

1. Open /v2/skills/admin -> Bundles -> filter "no_governance" or "below_min".
2. Click the bundle to open its detail panel – see SKILL.md, scripts/, intent, data_dependencies, file CHECKSUMS.
3. Run the **Activation Gate** (SPEC-0130 §5.1):
 - ≥5 test runs (or smoke-test marker)
 - DoD validated against frontmatter
 - human_checkpoints defined
 - context_scope declared
 - Security review for yellow / red
4. Optional sandboxed dry-install:

```
skillctl install <name>@<version> --home /tmp/review-sandbox-$$ --allow-yellow --verbose
```

5. Issue the verdict:

```
skillctl attest <bundle-digest> \
  --level green \
  --rationale "Activation gate passed; intent matches data_deps; smoke-tested locally." \
  --reviewer-id id:reviewer@m3c \
  --key ~/.claude/skillctl-keys/reviewer.key
```

You may NOT:

- Author bundles you review (separation of duties).
- Bypass the Activation Gate.
- Modify `_skill_bundles`, `_skill_signatures`, `_skill_identities` directly.

v1 limitation: in registry-on-behalf-of mode (CEO briefing Q3 lock) the registry key signs the attestation; this is logged but doesn't replace per-reviewer keys, which are a Q3-2026 deliverable. Customer engagements should document this in the security narrative.

5.3 CISO

Console layout (`/v2/ciso?tenant=<your-tenant>`):

Tab	What you do
Bundles	Browse all bundles in your tenant scope. Filter by governance, by source, by data dep.
Attestation queue	Process the SLA-tracked queue: Approve (attest green tenant-scoped) · Block (attest red tenant-scoped -> propagates to consumer-side <code>skillctl install --tenant <id> rejection</code> with exit 13) · Escalate (queue for next reviewer with SLA timer)
Data sources	Per-(skill, DataSource) chip: [authorize \ deny \ pending]. New (skill, data_source) pairs default to pending so you never silently inherit permissions.
Runtime (SPEC-0202)	Live tail of active capability tokens. Per row: token id, skill, caller, age, refusal count. [x] revoke (≤2s propagation via Kafka). [a] attenuate (modal: pick rules to add; child token issued).
Policy	Edit <code>_skill_runtime_policies</code> for your tenant: defaults, hard floors, per-skill overrides (see §5.3.2 below).
Audit	Quarterly export PDF/CSV. SPEC-0192 §8 acceptance criterion.

Per-skill review checklist (the panel SPEC-0196 §8 renders):

```
fetch-contract v1.0.0 (G)
Trust chain:  author -> registry -> reviewer [ok]
Capabilities: skill:claude-code/scaffolding · skill:document-generation
Data deps:
  READ:  ds:er1/plm/skill-creations          [authorized]
  WRITE: ds:filesystem/cwd                   [authorized]
  PASS:  ds:http/anthropic-api                [pending]   ← decide
Side-effects: fs:write, llm:call
```

```
Subprocess:  pandoc, git
Imported_from: (none)
Runtime envelope required: true
[Authorize]  [Deny]  [Quarantine]
```

The chip is the **decision handle**. Authorize by data dependency, not by skill alone. The next install on a consumer machine respects the verdict.

5.3.1 Block-by-source If a hub turns hostile:

Settings -> Source policies -> Block source

```
host: skillhub.club
effective: now
rationale: "Compromised author key reported 2026-05-07."
```

All bundles whose `imported_from.host` matches now fail install with exit 19. Replay-test the verdict via `skillctl invoke-replay (SPEC-0202)`.

5.3.2 Runtime policy template /v2/ciso?tenant=<id>&tab=policy-edit_skill_runtime_policies:

```
defaults:
  max_token_ttl_seconds: 300
  max_runtime_seconds: 120

hard_floors:
  egress_allowlist: [api.anthropic.com, registry.aims-core.example]
  subprocess_denylist: [curl, wget, ssh, rm, kubect!, gcloud]
  destructive_skills_require_dual_token: true

overrides:
- skill_name: deploy-stage
  max_runtime_seconds: 600
  egress_allowlist_add: [console.cloud.google.com]
```

Any envelope is (bundle declaration) $\cap$ (this policy) $\cap$ (caller request). Empty intersection -> no token issued, skill cannot run.

5.3.3 Quarterly audit binder Export from Audit tab -> PDF/CSV. Contains:

- Bundle inventory at quarter end
- Every Approve/Block/Escalate verdict by reviewer + timestamp
- Every (skill, data_source) authorization chip change
- Every runtime token issued/revoked/attenuated
- Every gate.refused event (envelope violations)
- Every imported bundle and its source chain

This is what auditors and regulators receive. Keep `tenant_scope` on every row – never include other tenants' data.

5.4 Registry Operator

Daily flow is mostly automated. Your scheduled work:

```
cd m3c-tools-maintenance/INFRA/skill-registry/
./bootstrap.sh --tenant <tenant-id> --env <prod|stage>
```

5.4.1 Provisioning a tenant Provisions: GCS bucket skill-bundles-<tenant>, Firestore namespace tenant_<id>, registry signing key in Secret Manager (semanpix/skill-registry-key-<tenant>), default trust-roots template at tenants/<id>/skill-trust-roots.yaml.

Distribute tenants/<id>/skill-trust-roots.yaml to all consumer machines for that tenant.

```
curl -X POST $REG/api/skills/identities \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "id": "id:author@<org>",
    "ed25519_pubkey": "<base64 SPKI>",
    "tier": "user",
    "tenant_scope": "<tenant-id>",
    "registered_by": "<your-id>",
    "rationale": "<why>"
  }'
```

5.4.2 Registering an authoring identity (v1 manual) v2 (post-2026-Q3) will move this to GitHub OIDC or SSO-backed identity registration.

5.4.3 Key rotation Follow CISO-WORK/RUNBOOK-skill-registry-key-rotation.md verbatim. The overlap-publish pattern: registry_keys[] carries multiple entries during the window; verifier accepts any non-retired entry. **Abrupt key replacement breaks every consumer's skill-trust-roots.yaml** – never skip the overlap window.

```
curl -X POST $REG/api/skills/identities/<id>/revoke \
  -H "Authorization: Bearer $ADMIN_TOKEN" \
  -d '{"effective_at": "<ISO>", "rationale": "<why>"}'
```

5.4.4 Identity revocation Triggers the cascade in §4.5 Author-key revocation.

You may NOT:

- Author or attest skills.
- Make per-tenant policy decisions (CISO's job).
- Modify author signatures or governance attestations directly.

5.5 Tenant Admin

Files in your scope:

tenants/<tenant-id>/	
├─ config.yaml	# tenant config + SLA tiers + notify-targets
├─ skill-trust-roots.yaml	# which registry keys this tenant accepts
├─ role-bindings.yaml	# tenant_ciso, tenant_admin, etc.
└─ runtime-policy.yaml	# SPEC-0202 envelope policy

```
curl -X POST $REG/api/tenants/<tenant-id>/role-bindings \
  -d '{"role": "tenant_ciso", "identity": "id:ciso@<org>"}'
```

5.5.1 Bind the CISO The bound identity gains access to /v2/ciso?tenant=<id>.


```
# tenants/<id>/config.yaml
sla:
  P0:
    review_within_hours: 4
    notify: ["#ciso", "ciso@<org>"]
  P1:
    review_within_hours: 24
  P2:
    review_within_hours: 168    # 1 week
notify_targets:
  slack: "https://hooks.slack.com/services/..."
  email: "ciso@<org>"
governance_minimum: green
```

5.5.2 Configure SLA tiers Policy attestations below the minimum trigger an SLA-tracked notification (SPEC-0192 §7).

5.5.3 Pause skill fan-out If something is on fire:

```
curl -X POST $REG/api/tenants/<tenant-id>/fanout-pause \
  -d '{"rationale": "incident-2026-05-07-001", "until": "<ISO>"}'
```

KafScale stops republishing to tenant.<id>.skills.* until you resume. Existing consumer installs keep working; new admissions wait.

You may NOT: attest, author, rotate keys, edit other tenants' configs.

5.6 End-User / Skill Consumer

```
# 1) Build/install skillctl (your operator distributes a signed binary; verify it)
which skillctl                                # /usr/local/bin/skillctl

# 2) Pin trust roots
mkdir -p ~/.config/m3c/skill-trust-roots
# (your operator gives you the registry pubkey; save it locally)
skillctl trust add \
  --registry https://aims.example.com/api/skills \
  --pubkey ~/.config/m3c/skill-trust-roots/aims-example.pub \
  --id aims-example-prod

skillctl trust list                            # confirm
```

5.6.1 First-machine setup The trust-roots file is at ~/.claude/skill-trust-roots.yaml.

```
skillctl install didactic-session@1.0.0 --tenant <your-tenant>
# or, with a yellow override (audited):
skillctl install some-skill@2.0.0 --allow-yellow

# Verify any installed skill (re-runs the chain check)
skillctl verify didactic-session
```

5.6.2 Install a skill Verifier order (SPEC-0188 §7): digest -> author sig -> registry sig -> governance ≥ minimum -> tenant policy -> data_dependencies all authorized -> depends_on resolved -> atomic move to ~/.claude/skills/<name>/. Any failure = no write. Numbered exit codes §8.

```
skillctl run didactic-session # default envelope
skillctl run didactic-session --tenant <your-tenant> # tenant-scoped
skillctl run didactic-session --max-runtime 600 # request larger envelope (interse
```

5.6.3 Run a skill The wrapper:

1. POSTs /api/skills/capability-tokens/issue – gets a 5-minute signed token.
2. Sets M3C_SKILL_CAPABILITY_TOKEN=<token> in the child env.
3. LD_PRELOAD/DYLD_INSERT_LIBRARIES injects the gateway shim.
4. SOCKS sentinel steers all egress through a tenant proxy that requires the live token_id as auth.
5. Skill runs. Every HTTP/subprocess/file call is gated. Refusals -> exit 30-39 + audit row.
6. invocation.completed lands on m3c.<tenant>.skill_invocations.

You almost never see the envelope – only when you violate it.

```
skillctl audit # OK / UNVERIFIED / BROKEN / BELOW
skillctl audit --json | jq # for scripts
skillctl audit --cleanup --dry-run-cleanup # preview deletions
skillctl audit --cleanup --confirm-delete # apply
```

5.6.4 Audit your machine Run weekly (or in pre-commit / CI). BELOW_MIN skills are reported but not auto-deleted – you may install with --allow-yellow instead.

You may NOT:

- Edit a skill in ~/.claude/skills/ and expect it to keep its trust state – skillctl audit flags it BROKEN.
- Bypass tenant-scoped block verdicts (exit 13).
- Publish to the registry (Author’s job).

5.7 Auditor

Read-only forensics. No write authority anywhere. Independence is the value – never also be Reviewer or CISO for the same scope.

What you read:

Source	What’s in it
_skill_audit_log	Every state change with actor, timestamp, before/after
_skill_attestations	All governance attestations (current + historical)
_skill_signatures	Author + registry signatures
_skill_graph_edges	Structural + semantic edges
m3c.<tenant>.skill_invocations Kafka topic	Every issue, attenuate, revoke, gate.allowed, gate.refused, invocation.completed
skillctl audit JSON output across machines	Per-machine truth
~/.cache/m3c/imports/<sha256>/scan-report.json (per machine)	Pre-airlock scanner output

Reconstructing a single skill's chain:

```
# Bundle row + signatures
curl $REG/api/skills/by-digest/<digest>
curl $REG/api/skills/<digest>/signatures
curl $REG/api/skills/<digest>/attestations

# Identity provenance
curl $REG/api/skills/identities/<author-id>
curl $REG/api/skills/identities/<author-id>/history      # rotations + revocations

# Audit log chronology
curl $REG/api/skills/audit-log?bundle_digest=<digest>    # ordered events

# Runtime invocations (per tenant)
kafka-console-consumer --topic m3c.<tenant>.skill_invocations --from-beginning \
  | jq 'select(.bundle_digest == "<digest>")'

# Local install state on a machine
ssh <host> 'skillctl audit --json' | jq '.skills[] | select(.bundle_digest == "<digest>")'
```

Issuing findings: you cannot enforce. Findings escalate to:

- CISO (block) for tenant-scoped policy
- Reviewer (re-attest) for governance verdict change
- Registry Operator (revoke identity) for key compromise

5.8 m3c Consultant overlay

Per-role consulting deliverables (from ROLES.md §4):

Role	Consultant deliverable
Author	Author handbook tailored to customer stack + 5 reference skills published in their tenant
Reviewer	Reviewer playbook with N concrete examples graded against customer policy
CISO	Tenant policy rule-set v1 + CISO Console training session + the 7-criterion sign-off
Registry Operator	Tenant bootstrap delivered + key rotation runbook localized + first dry-run rotation completed
Tenant Admin	Tenant config v1 in git + role binding playbook
End-User	Per-user install kit + SKILLOR-WORK-style first-run session log per machine
Auditor	Quarterly audit template + dashboard + reproducible forensics runbook

Exit criterion: customer's roles execute the four interaction patterns (§4) without consultant intervention for one full quarter.

6. Decision tree: which command should I run?

"I want to..."

... see what skills are on this machine	-> skillctl scan	
... see which are signed/verified/broken	-> skillctl scan --with-trust	
... compare two scans	-> skillctl diff	
... view a scan in browser	-> skillctl review	
... open the read-only TUI inventory	-> skillctl browse	
... freeze current scan as baseline	-> skillctl seal	
... fix annotation gaps in a baseline	-> skillctl consolidate	
... pack a skill into a .skb	-> skillctl pack <skill-dir>	
... generate a signing keypair	-> skillctl keygen --out <path>	
... sign a bundle	-> skillctl sign <bundle.skb> --key <path>	
... verify a bundle's author signature locally	-> skillctl verify-sig <bundle.skb> -	
-pubkey <path>		
... run the ready-to-promote gate + push	-> skillctl propose <name>	[SPEC-0194]
... pull metadata of locally-scanned skills		
into the registry without uploading bytes	-> skillctl awareness sync	[SPEC-0195]
... read back what was admitted in a session	-> skillctl awareness verify	
... reset a session's awareness admissions	-> skillctl awareness reset --	
session <tag>		
... declare or update a skill's intent +		
data_dependencies (SPEC-0196) without		
touching code	-> skillctl intent declare <name>	
... import a skill from a public hub	-> skillctl import-	
public <ref>		[SPEC-0201]
... list staged imports	-> skillctl import-list	
... lint the import policy	-> skillctl import-policy lint <path>	
... pin a registry as a trust root	-> skillctl trust add	
... list pinned trust roots	-> skillctl trust list	
... unpin a registry	-> skillctl trust remove	
... install a verified skill	-> skillctl install <name>@<version>	
... re-verify an already-installed skill	-> skillctl verify <name>	
... run an installed skill (envelope-bounded)	-> skillctl run <name>	[SPEC-0202]
... replay an invocation under new policy	-> skillctl invoke-replay <token-id>	
... sign a governance verdict	-> skillctl attest <digest> --	
level <green yellow red>		
... antivirus-style audit of all installed		
skills	-> skillctl audit	
... clean up broken/unverified installs	-> skillctl audit --cleanup	
... sync local skill-usage telemetry to		
aims-core	-> skillctl sync-usage	

When in doubt, see the full reference: [SKILLCTL-MANUAL.md](#).

7. Maturity model

Per ROLES.md §5 and CONCEPT.md §6.2 – three tiers per role and per process. Maturity refers to **operational practice**, not feature completeness.

Tier	Inventory	Admission	Attestation	Audit	Authorization	Runtime
Bronze	skillctl scan produces JSON; no registry	-	-	-	-	-
Silver	Awareness sync to a tenant registry; bundles admitted	Awareness path live	Default attestations only (yellow user-tier, green plugin-tier)	skillctl audit weekly per machine	-	-
Gold	skillctl propose is daily flow; gate enforces frontmatter discipline	Author path live (full chain)	Named human reviewer attestations; CISO Console operational	Audit verdict in deploy gate (CI/pre-commit)	Per-(skill, data_source) authorization with policy engine	skillctl run envelope-bounded; live re-voke/attenuate
Platinum	Cross-host inventory aggregation; KafScale fan-out	Multi-tenant; KafGraph backend	N-of-M reviewer requirement; per-reviewer keys	Behavioral attestation (sandbox runs catch lying intent)	Cross-skill composition policy (graph walk)	Federated runtime SLAs; signed end-to-end invocation manifests

The maturity tiers double as **billable engagement scopes**:

Phase	Target	Effort	Unlocks
Bronze->Silver	tenant provision	~1-2 weeks	inventory + registry on customer infra
Silver->Gold	CISO process + audit gate	~3-4 weeks	the saleable artifact (CISO sign-off process)
Gold->Platinum	multi-tenant scale-out	ongoing	the federated/regulated tier

8. Exit codes

Centralised in cmd/skillctl/exit.go. Branching on these is safe; parsing log lines is not.

8.1 Generic

Code	Name	Meaning
0	OK	Success
1	generic	Other error (read stderr)
2	usage	Usage / flag error
3	(reserved – audit BROKEN)	At least one skill audited as broken

8.2 SPEC-0188 §11 – bundle / install / verify (10..16)

Code	Name	Meaning
10	digest_mismatch	Recomputed digest ≠ bundle name digest
11	author_sig_invalid	<digest>.author.sig failed verification
12	registry_not_pinned	Registry not in ~/.claude/skill-trust-roots.yaml
13	governance_below_minimum	Attested level < trust-root or tenant minimum (use --allow-yellow to override per-install, audited)
14	data_dep_denied	Tenant CISO denied a (skill, data_source) chip
15	depends_on_unsatisfied	A depends_on skill failed to install
16	revoked	Registry attestation says revoked

8.3 SPEC-0201 – upstream import airlock (17..19, 4..5)

Code	Name	Meaning	Fix
4	pin_required	Fetches without @sha256:...	Re-run with the printed pin
5	scanner_refuse	Static scan finding above floor	Read scan-report.json; either fix upstream or get policy exception
17	no_source_policy	Policy file missing/empty/host-not-allowed	Edit ~/.claude/skill-import-policy.yaml
18	upstream_intent_capped	Tried propose --intent green on imported bundle	Use --intent yellow; reviewer attests (G) separately
19	source_blocked	Tenant CISO blocked this source	Talk to the CISO; verdict id in stderr

8.4 SPEC-0195 §11 – awareness / intent (18..19)

Code	Name	Meaning
18	intent_inconsistent	intent and data_dependencies cross-validation failed
19	identity_mismatch	Author identity does not match registered identity

(Codes 18/19 are shared between SPEC-0195 and SPEC-0201 by design – both indicate boundary-policy violations at admission time.)

8.5 SPEC-0202 – runtime envelope (30..39)

Code	Name	Meaning
30	envelope_capability_missing	Skill tried an op (e.g. subprocess_run) not in the envelope
31	envelope_data_source_missing	Skill touched a data source not declared / not allowed
32	egress_host_not_allowed	HTTP request to a host outside egress_allowlist
33	subprocess_denied	Tried to exec something on subprocess_denylist
34	file_outside_envelope	Read/write to a path outside declared data_sources
35	token_expired	TTL elapsed mid-call
36	token_revoked	CISO killed the token
37	token_invalid_signature	Token bytes tampered
38	runtime_quota_exceeded	max_runtime_seconds hit
39	egress_byte_quota_exceeded	max_egress_bytes hit

Plus issuer-side HTTP refusals: 403 governance_below_minimum, 403 envelope_empty, 403 source_blocked, 503 issuer_overloaded.

9. Cookbook

9.1 Onboard a new author on a fresh laptop

```
# 1) Get skillctl
curl -L https://aims.example.com/dist/skillctl-darwin-arm64 -o /usr/local/bin/skillctl
chmod +x /usr/local/bin/skillctl
skillctl --help

# 2) Generate a signing keypair (keep both files local; only .pub leaves the laptop)
mkdir -p ~/.claude/skillctl-keys
skillctl keygen --out ~/.claude/skillctl-keys/author

# 3) Send the .pub to your Registry Operator (Slack/email/issue tracker)
cat ~/.claude/skillctl-keys/author.pub
```

```
# 4) Pin the registry trust root they hand back
skillctl trust add --registry https://aims.example.com/api/skills \
  --pubkey ~/.config/m3c/skill-trust-roots/aims-prod.pub \
  --id aims-example-prod

# 5) Verify with a known-good bundle
skillctl install didactic-session@1.0.0 --tenant <your-tenant>
skillctl run didactic-session
```

9.2 First skill ever

```
# Lay it down
mkdir -p ~/.claude/skills/hello-skill
cat > ~/.claude/skills/hello-skill/SKILL.md <<'EOF'
---
name: hello-skill
version: 0.1.0
governance_level: yellow
owner: id:author@<org>
human_checkpoints: ["confirm-output"]
context_scope: ["fs:write:<cwd>/output/**"]
---

# hello-skill
Writes a hello.txt to ./output/.
EOF
mkdir -p ~/.claude/skills/hello-skill/scripts

# Declare intent + data deps
cat > ~/.claude/skills/hello-skill/bundle.json <<'EOF'
{
  "schema": "m3c-skill-bundle/v1",
  "name": "hello-skill",
  "version": "0.1.0",
  "summary": "Writes a hello.txt.",
  "author_governance_intent": "yellow",
  "author_governance_rationale": "Writes one local file.",
  "intent": {
    "summary": "Writes a hello.txt to ./output/.",
    "side_effects": ["fs:write"],
    "destructive": false,
    "network": false,
    "subprocess": [],
    "claims": []
  },
  "data_dependencies": [
    {
      "id": "ds:filesystem/cwd", "kind": "local_fs",
      "access": "write", "scope": "<cwd>/output/**",
      "reason": "Write hello.txt."
    }
  ],
  "runtime_envelope_required": true
}
```


EOF

```
# Propose
skillctl propose hello-skill --intent yellow --rationale "First skill."
```

9.3 Triage a “BELOW_MIN” skill across the team

```
# 1) Find affected machines (each person runs):
skillctl audit --json | jq '.skills[] | select(.verdict == "BELOW_MIN")' > /tmp/below_min.json

# 2) Find the cause -- likely a (Y) attestation against a (G)-required tenant
curl $REG/api/skills/by-digest/<digest>/attestations

# 3) Either:
#   a) Reviewer re-attests upward:
skillctl attest <digest> --level green --rationale "<reason>" --reviewer-id <id> --key <path>
#   b) Or: tenant relaxes governance_minimum (CISO Console -> Policy)
#   c) Or: end-users install with --allow-yellow (audited, individual decision)
```

9.4 Investigate a gate.refused event

```
# 1) From the CISO Runtime tab, copy the token id and refusal row
TOKEN=ct:01H...mz9
EXIT_CODE=32 # egress_host_not_allowed

# 2) Pull the full invocation chain
kafka-console-consumer --topic m3c.<tenant>.skill_invocations --from-beginning \
  | jq "select(.token_id == \"\$TOKEN\")"

# 3) Replay under the same policy to confirm determinism
skillctl invoke-replay $TOKEN --mode dry

# 4) If you want to tighten policy further, replay under the new policy
skillctl invoke-replay $TOKEN --mode live --tenant <tenant>

# 5) If the skill has a legitimate need, edit the runtime policy (CISO):
#   overrides[].egress_allowlist_add += <host>
```

9.5 Quarterly audit binder generation

```
# 1) Trigger the export from the CISO console (Audit tab -> Export quarter)
# 2) Or script it:
curl -X POST $REG/api/tenants/<tenant>/audit/export \
  -d '{"from": "2026-01-01", "to": "2026-03-31", "format": "pdf"}' \
  -H "Authorization: Bearer $TOKEN" \
  --output Q1-2026-skill-audit-<tenant>.pdf

# 3) Cross-check the inventory
skillctl audit --json > Q1-2026-machine-<host>.json # per machine; collect
```

10. Failure modes & remediation

Symptom	Likely cause	Remediation
firestore.exceptions.PermissionDenied on awareness sync	Wrong bundles chain selected	Set GOOGLE_APPLICATION_CREDENTIALS_FIRESTORE to the youtubeanalyzer23 service-account key Build via <code>cd m3c-tools && go build -o /usr/local/bin/skillctl ./cmd/skillctl</code> ; or set SKILLCTL_BIN
skillctl: command not found	Binary not on PATH	Operator distributes the registry pubkey; skillctl trust add Bounce the consumer; or skillctl install --tenant <id> to force tenant resolution
skillctl install exits 12 (registry_not_pinned)	The bundle's registry isn't in trust-roots	Check container logs for graph-edge-failed warnings; if it recurs, file BUG-NNNN with the warning text
skillctl install exits 13 even after re-attest	Tenant policy still has the old value cached	Make intent and data_dependencies consistent and re-run
Trust Graph empty in admin UI even though Bundles tab shows entries	Graph-edge writer didn't run (BUG-0131 territory)	Either the skill is buggy (fix it), or CISO needs to add the host to the tenant policy
skillctl pack exits 18 intent_inconsistent	Cross-validation found intent.network=false but a data_dep declares an HTTP source (or similar)	Re-run with --help-advanced to see --allow-overwrite (logs collisions but proceeds)
skillctl run exits 32 (egress_host_not_allowed)	Skill tried to reach a host not in tenant egress_allowlist	Confirm container env FIRESTORE_PROJECT_ID=youtubeanalyzer23; recompose with /dev-cycle
Skill counted in inventory but no bundle in admin UI after awareness sync	Ingest skipped due to digest collision (existing different-name doc)	Re-issue: skillctl run will mint a new one. For long-running, request --max-runtime <N>
Container shows "0 bundles" after run	Firestore project mismatch	Either revert the edit, or re-author through skillctl propose (don't mutate installed bundles)
skillctl run hangs / SOCKS sentinel rejects	Token expired (default 5 min)	
Pre-commit hook fails on ~/.claude/skills/ edit	skillctl audit flags BROKEN (file modified post-install)	

11. References

- **Project docs:** [README.md](#) · [CONCEPT.md](#) · [ROLES.md](#) · [GAP-ANALYSIS.md](#)
- **Operator briefs:** [IMPORT-AIRLOCK.md](#) (SPEC-0201) · [RUNTIME-ENVELOPE.md](#) (SPEC-0202)
- **Live procedure:** [SKILLOR-WORK/s1/USER-MANUAL.md](#)
- **CLI reference:** [SKILLCTL-MANUAL.md](#)
- **SPECs:** [-0114](#) · [-0115](#) · [-0121](#) · [-0122](#) · [-0130](#) · [-0188](#) · [-0189](#) · [-0190](#) · [-0192](#) · [-0193](#) · [-0194](#) · [-0195](#) · [-0196](#) · [-0197](#) · [-0198](#) · [-0200](#) · [-0201](#) · [-0202](#)
- **Governance handbook:** [.claude/GOVERNANCE.md](#) (active SPEC-0130)

- **Field bugs (the friction that shaped the design):** [BUG-0131](#) · [BUG-0132](#) · [BUG-0133](#)